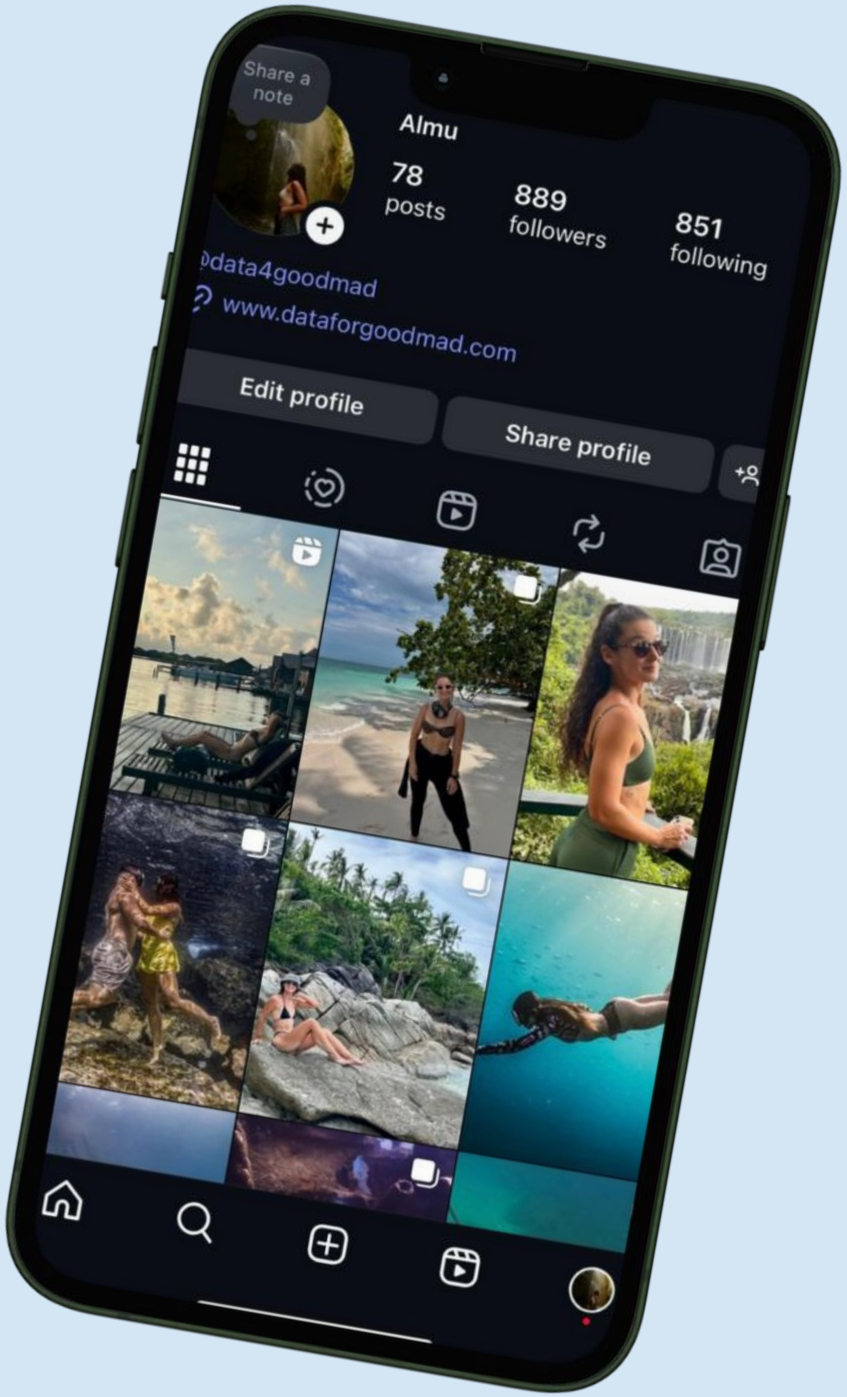
Almudena Barreiro

Did we break up? Auditing Instagram data with Python

in/almudena-barreiro

github.com/AlmuBC/instagram-behavior-analysis



01 - Who am I?

Nature lover, scientific mindset, sceptical, into adventure sports, from Galicia-Spain, not very popular, not very active on social media, I talk about books and travel

- May 2021: I leave Madrid and become a digital nomad
- Spring 2021: Lanzarote
- Summer 2021: Mexico
- September 2021: Changed jobs
- October 2021: Sitges
- September 2022: Bali
- April 2023: Bali - Divemaster
- May 2024: Bali
- January 2025: Uruguay-Brazil-Argentina
- March 2025: Bali

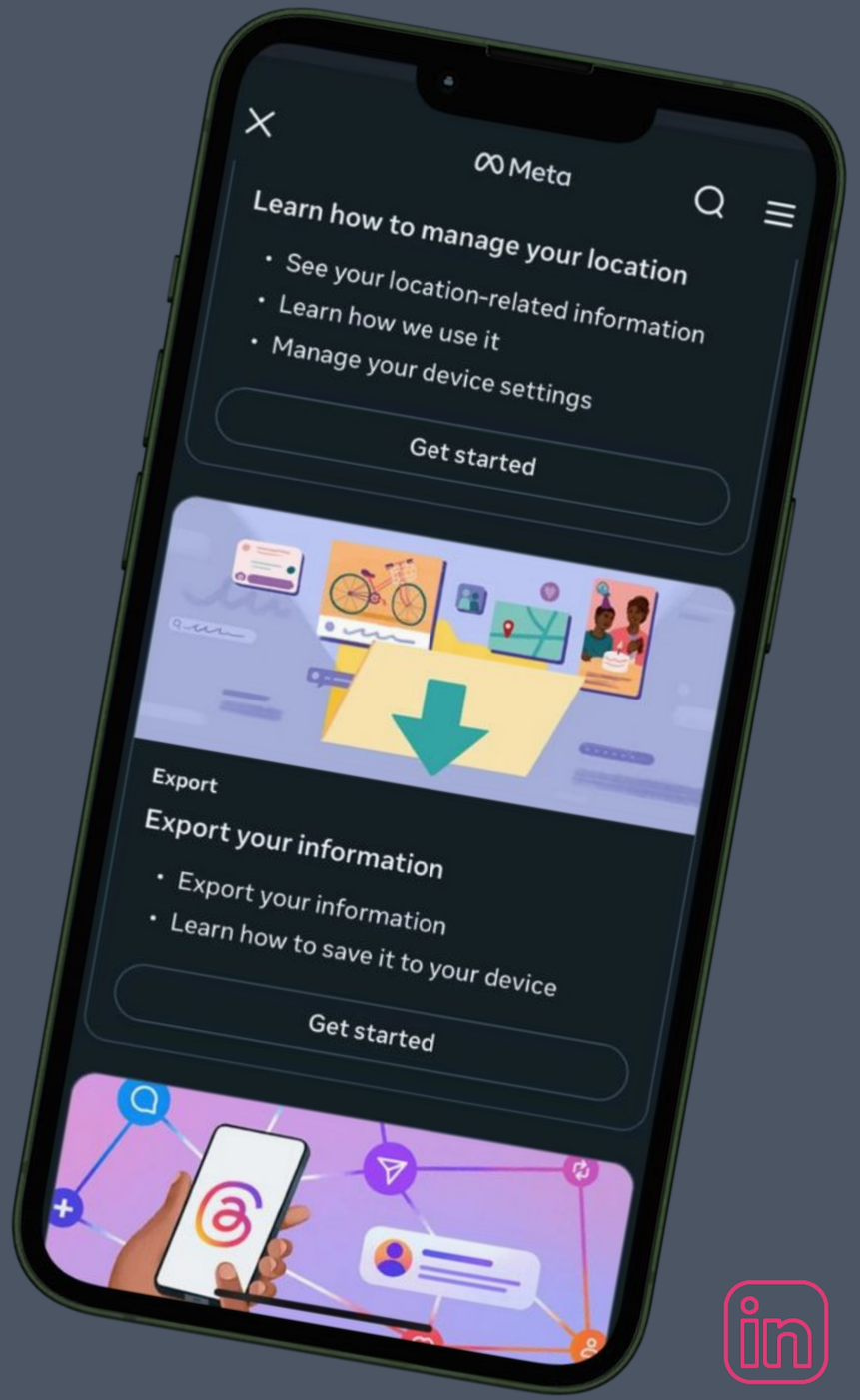
02 - Instagram and your data

You can request your Instagram data from your account, both in the app and on the web.

There are **two options:** a basic download and a more complete one called “specific information”. It takes a few days (sometimes up to a week) to prepare.

You can request it once or schedule it scheduled to be repeated.

You can find it under Settings and activity → Privacy Center → Export your information.

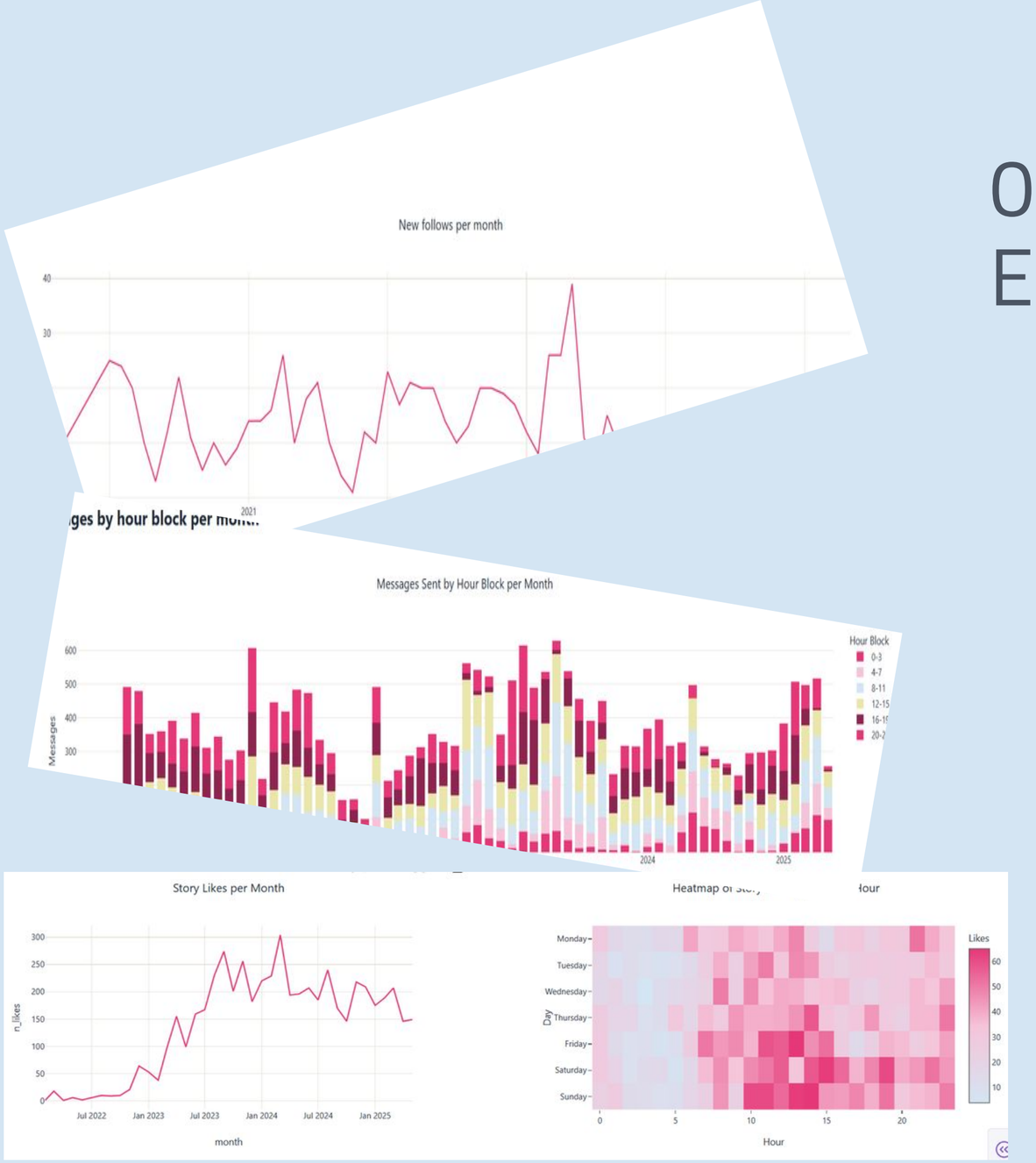


02 - Instagram and your data

What data can we find in the **“specific information”:**

- All messages sent and received, even if the person is blocked or has blocked us
- Everything we have posted in posts and stories
- All the likes we have given and to whom
- All ads shown to us and which ones we clicked





02 - Instagram and your data: EDA

Dash is a Python framework for building interactive dashboards.
We use it to visualise Instagram activity, but its biggest advantage (for me) is that it **allows models to be executed directly,** something that is not as straightforward in Power BI or Tableau.
Project structure:

- app.py: launches the app
- layout.py: defines the layout
- callbacks.py: interactive logic
- utils.py: utility functions
- assets/: styles and colours

02 - Instagram and your data: EDA



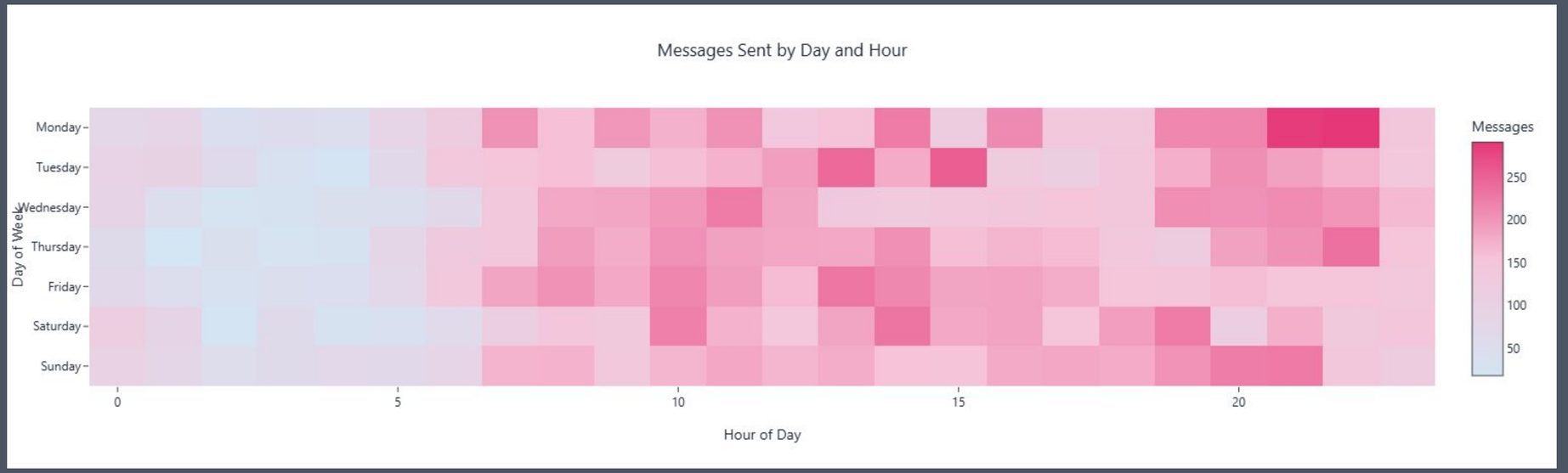
02 - Instagram and your data: EDA

- May 2021: I leave Madrid and become a digital nomad
- Spring 2021: Lanzarote
- Summer 2021: Mexico
- September 2021: Changed jobs
- October 2021: Sitges
- September 2022: Bali
- April 2023: Bali - Divemaster
- May 2024: Bali
- January 2025: Uruguay-Brazil-Argentina
- March 2025: Bali

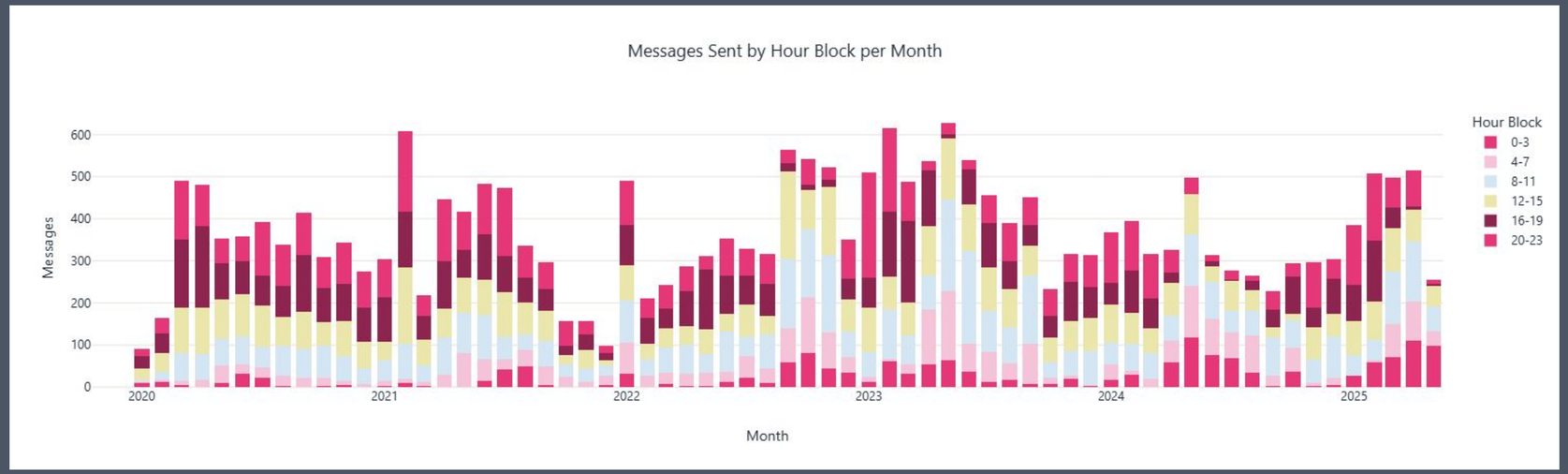


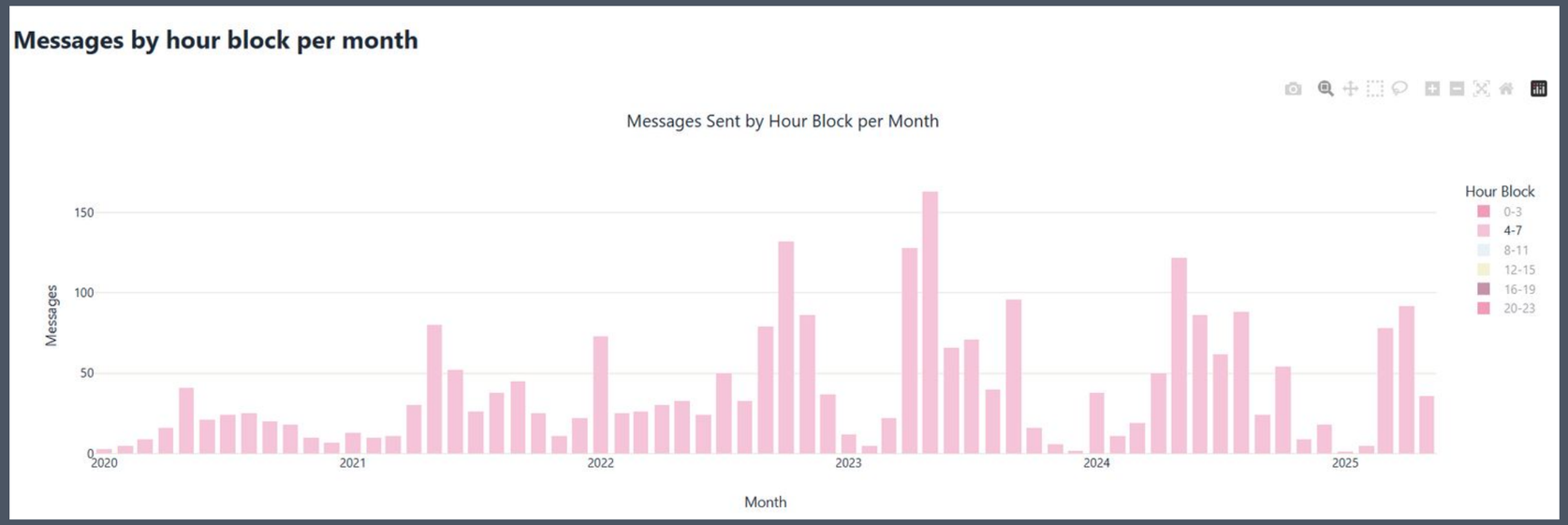
02 - Instagram and your data: EDA



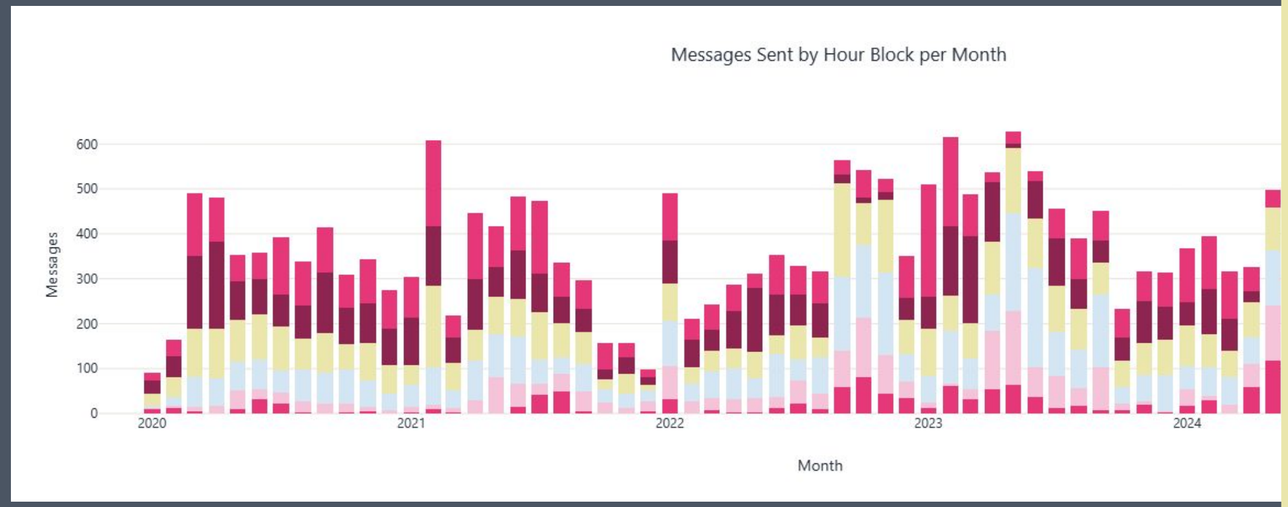


02 - Instagram and your data: EDA

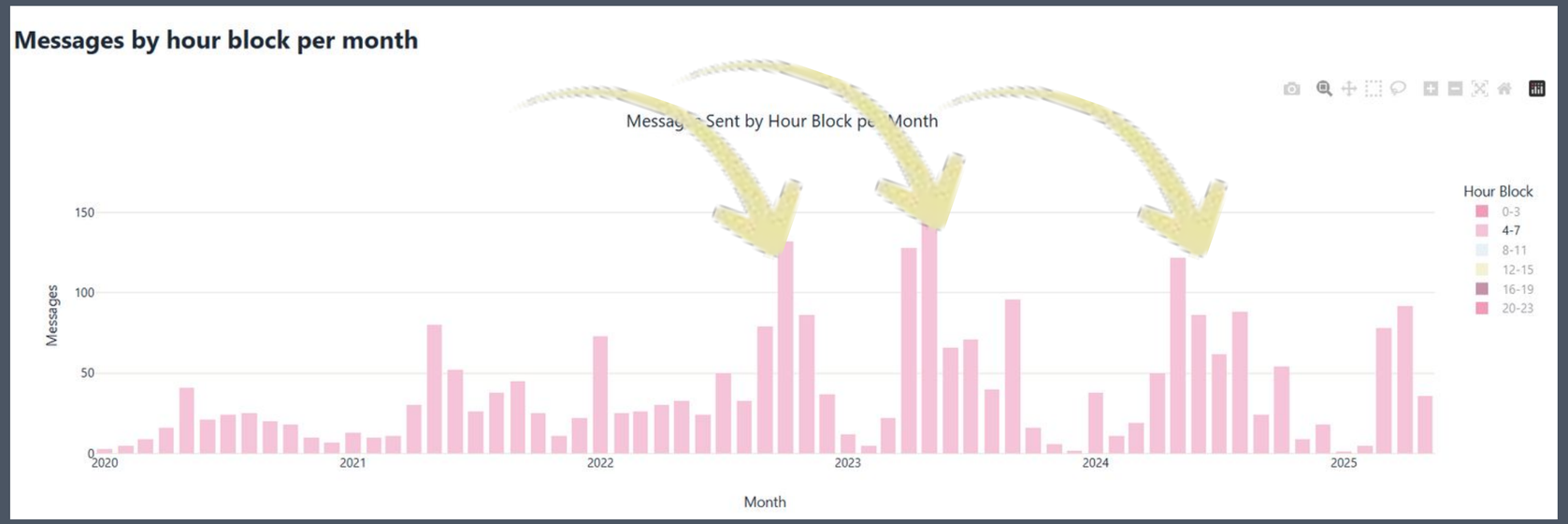




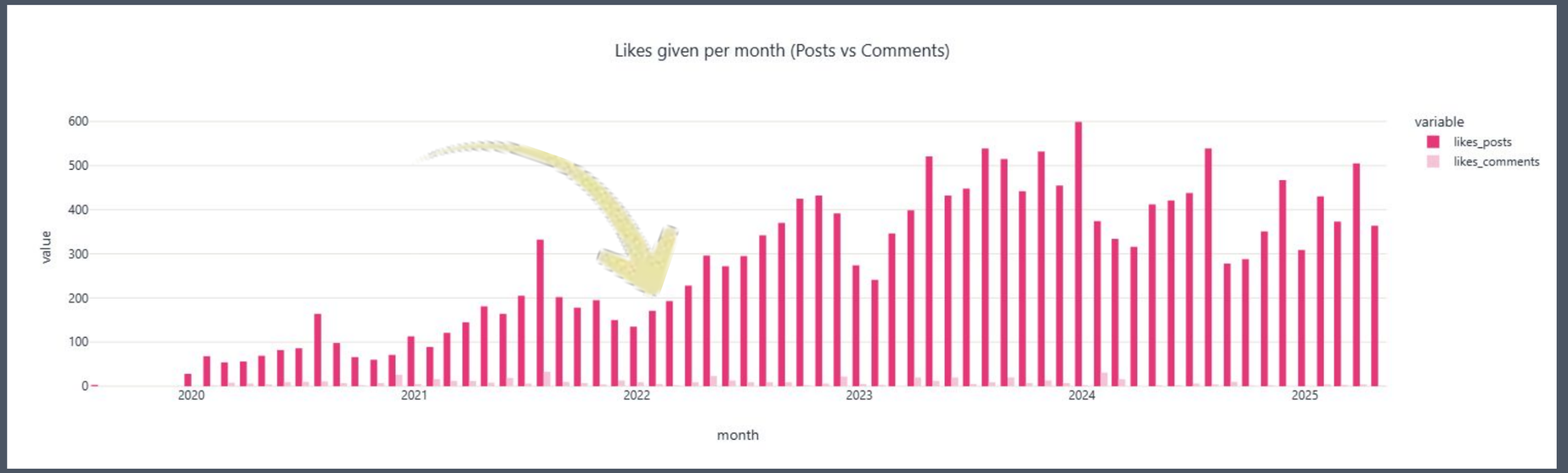
02 - Instagram and your data: ED

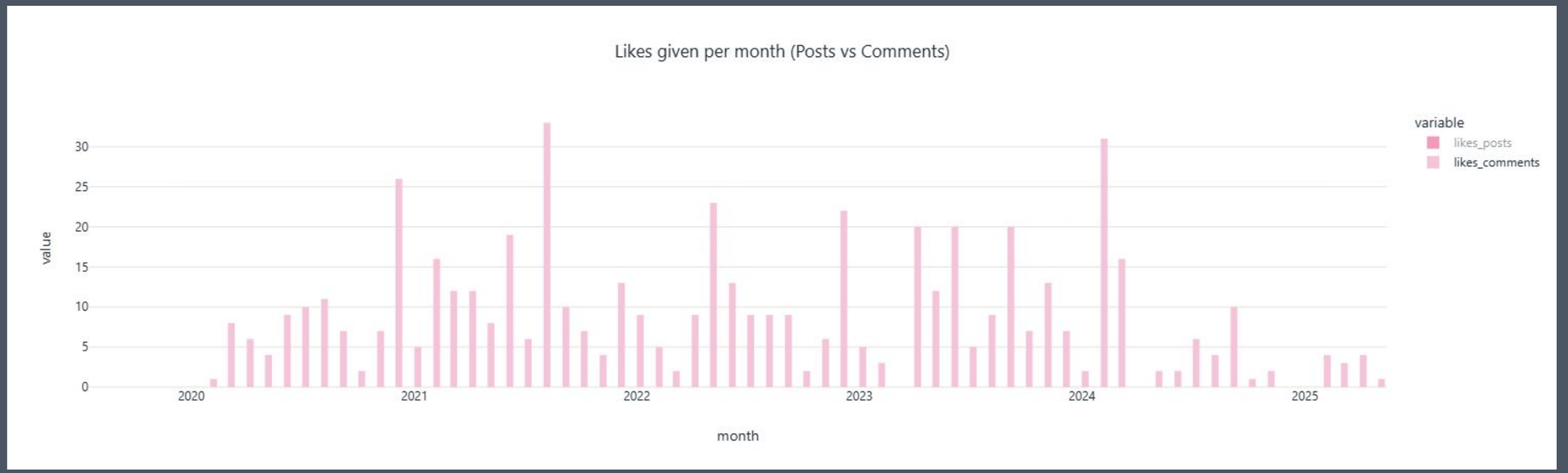


- May 2021: I leave Madrid and become a digital nomad
- Spring 2021: Lanzarote
- Summer 2021: Mexico
- September 2021: Changed jobs
- October 2021: Sitges
- September 2022: Bali
- April 2023: Bali - Divemaster
- May 2024: Bali
- January 2025: Uruguay-Brazil-Argentina
- March 2025: Bali



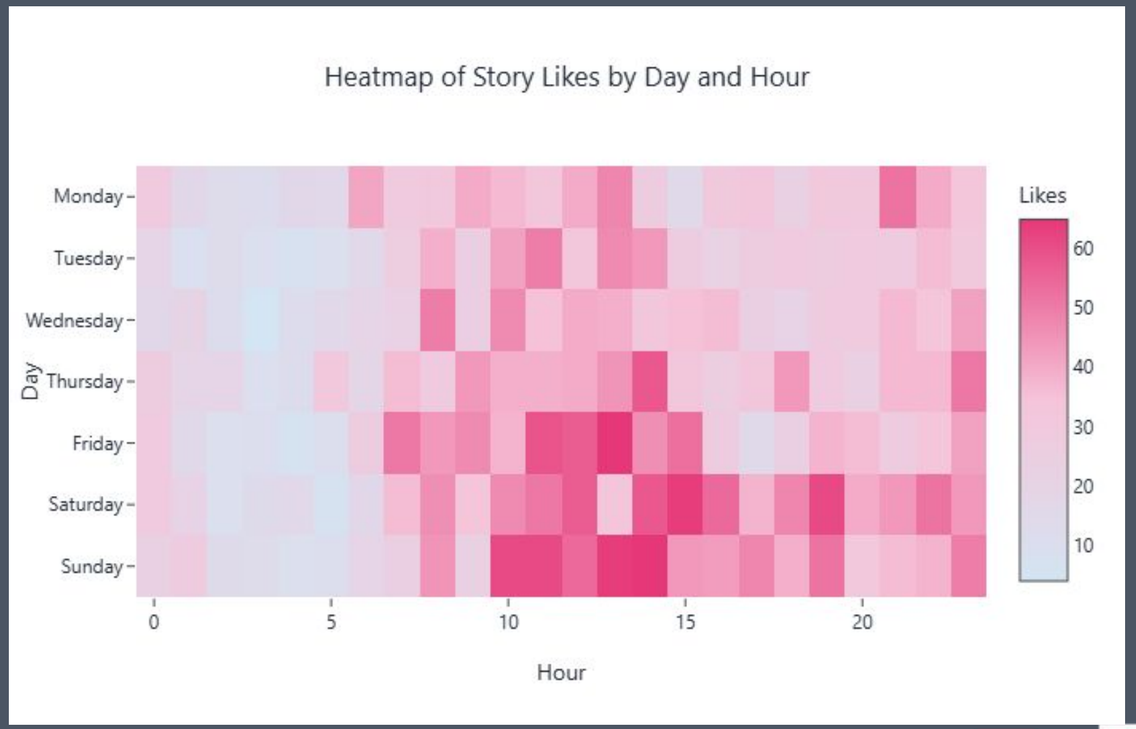
02 - Instagram and your data: EDA





02 - Instagram and your data: EDA





02 - Instagram and your data: EDA



02 - Instagram and your data: EDA




```
{
  "topics_your_topics": [
    {
      "title": "",
      "media_map_data": {

      },
      "string_map_data": {
        "Name": {
          "href": "",
          "value": "Fish",
          "timestamp": 0
        }
      }
    },
    {
      "title": "",
      "media_map_data": {
        |
      },
      "string_map_data": {
        "Name": {
          "href": "",
          "value": "Basketball",
```

MY BOSS

- Buceo
- Running
- Libros



MY BEST FRIEND

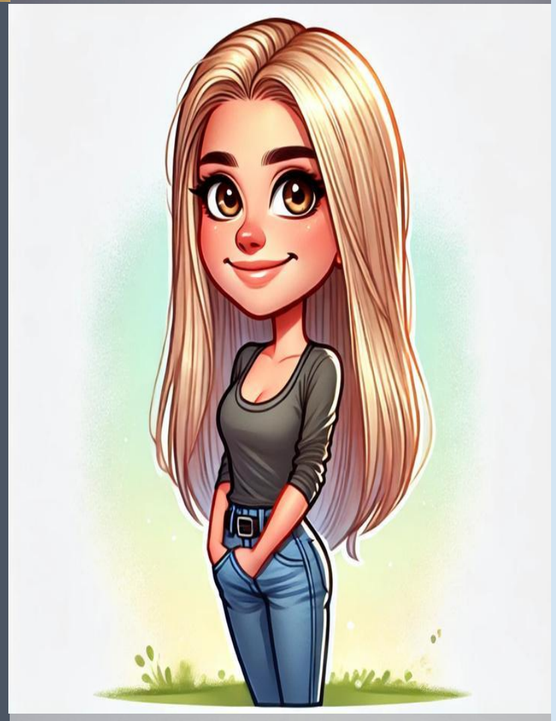
- Lectura
- Viajes
- Deportes

Who knows me best?



MY BOYFRIEND

- Libros
- Gimnasio
- Divulgación



MY BOSS

- Buceo
- Running
- Libros



MY BEST FRIEND

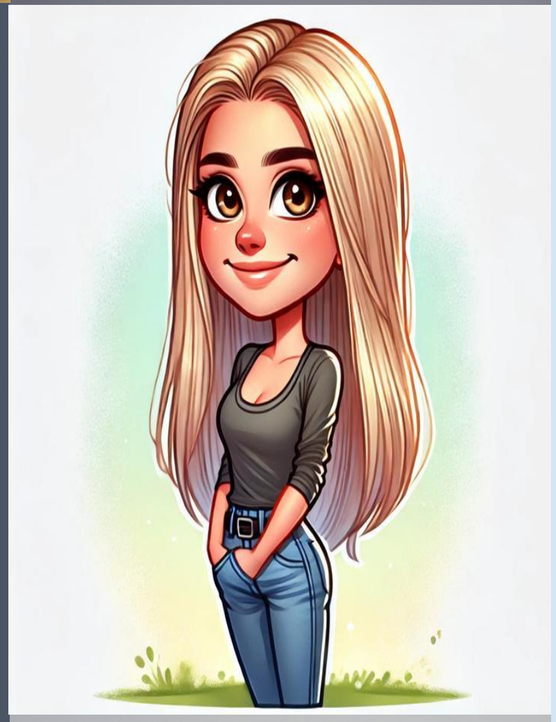
- Lectura
- Viajes
- Deportes

Who knows me best?



MY BOYFRIEND

- Libros
- Gimnasio
- Divulgación



How many topics to pick:

5

Pick Random Topics

Water Activities | Jewelry | Foods | Hair Care | Latin American Food

MY BOSS

- Buceo
- Running
- Libros



MY BEST FRIEND

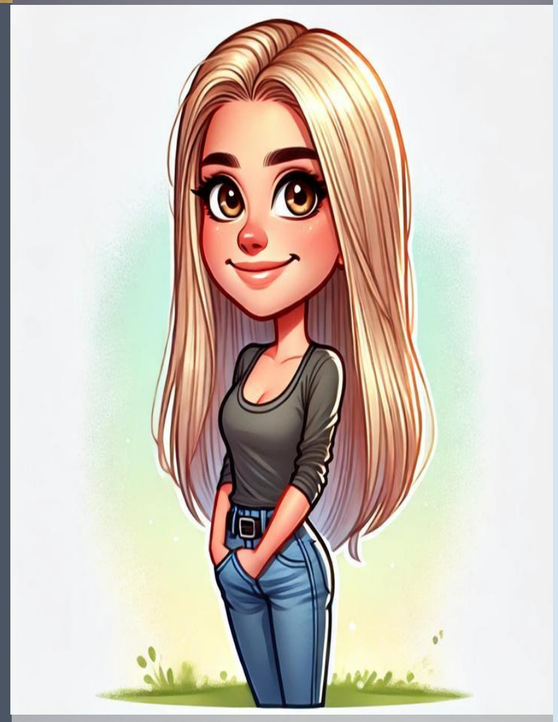
- Lectura
- Viajes
- Deportes

Who knows me best?



MY BOYFRIEND

- Libros
- Gimnasio
- Divulgación



How many topics to pick:

5

Pick Random Topics

Turtles | Beer | Nails & Nail Care | Drinks | Running

02 - Instagram and your data: EDA conclusions





- Travel periods are visible; the behaviour pattern clearly changes
- A small change is noticeable in 2022
- Repeated behaviours can be observed over the years

03 - Anomaly Detection

- **Events**: we define as an event any interaction we have with Instagram:
  - sent messages
  - likes on posts and stories

- For **anomaly detection**, our time series is defined as follows:
  - ds: the time component, aggregated hourly
  - y represents the events

	ds	# y
	Missing: 0 (0%) Distinct: 50506 (100%) Min: 2019-08-17 20:00:00 Max: 2025-05-22 05:00:00	Missing: 0 (0%) Distinct: 50 (<1%) Min 0 Max 88
0	2019-08-17 20:00:00	4
1	2019-08-17 21:00:00	1
2	2019-08-17 22:00:00	0
3	2019-08-17 23:00:00	0
4	2019-08-18 00:00:00	0
5	2019-08-18 01:00:00	0
6	2019-08-18 02:00:00	0
7	2019-08-18 03:00:00	0
8	2019-08-18 04:00:00	0

03 - Anomaly Detection

Rolling z-score: We measure how unusual a value is compared to what normally happens at that same time slot (for example, during the previous week). If it deviates significantly from normal behaviour, we flag it as a potential anomaly.

```python
def rolling_zscore_anomalies(
    ts_df: pd.DataFrame,
    y_col: str = "y",
    window: int = 24 * 7,   # 1 week at hourly frequency
    min_periods: int = 24,
    z_thresh: float = 3.0
) -> pd.DataFrame:
    """
    Compute rolling z-score over a univariate series and flag anomalies where |z| > z_thresh.
    Expects columns ['ds', y_col] and a regular time frequency.
    Returns a copy of ts_df with columns: ['roll_mean','roll_std','zscore','is_anomaly_z'].
    """
    df = ts_df.sort_values("ds").copy()
    s = df[y_col].astype(float)

    roll_mean = s.rolling(window=window, min_periods=min_periods).mean()
    roll_std = s.rolling(window=window, min_periods=min_periods).std(ddof=0)

    # Avoid division by zero
    roll_std_safe = roll_std.replace(0, np.nan)
    z = (s - roll_mean) / roll_std_safe

    df["roll_mean"] = roll_mean
    df["roll_std"] = roll_std
    df["zscore"] = z
    df["is_anomaly_z"] = (z.abs() > z_thresh).fillna(0).astype(int)
    return df
```

03 - Anomaly Detection

Isolation Forest: learns what your normal Instagram behaviour looks like (for example, how often you interact depending on the hour or day).

It then looks for moments that deviate strongly from that pattern and flags them as potential anomalies.
It is a more powerful approach because it considers context: giving 20 likes on a Tuesday morning is not the same as giving 20 likes on a Saturday night.

```
df = ts_df.sort_values("ds").copy()
df["ds"] = pd.to_datetime(df["ds"])

X = pd.DataFrame(index=df.index)
X["y"] = df[y_col].astype(float)

if with_context:
    X["hour"] = df["ds"].dt.hour
    X["dow"] = df["ds"].dt.dayofweek
    # Simple lags and diffs
    X["lag1"] = X["y"].shift(1)
    X["lag24"] = X["y"].shift(24)
    X["diff1"] = X["y"].diff(1)
    X["diff24"] = X["y"].diff(24)

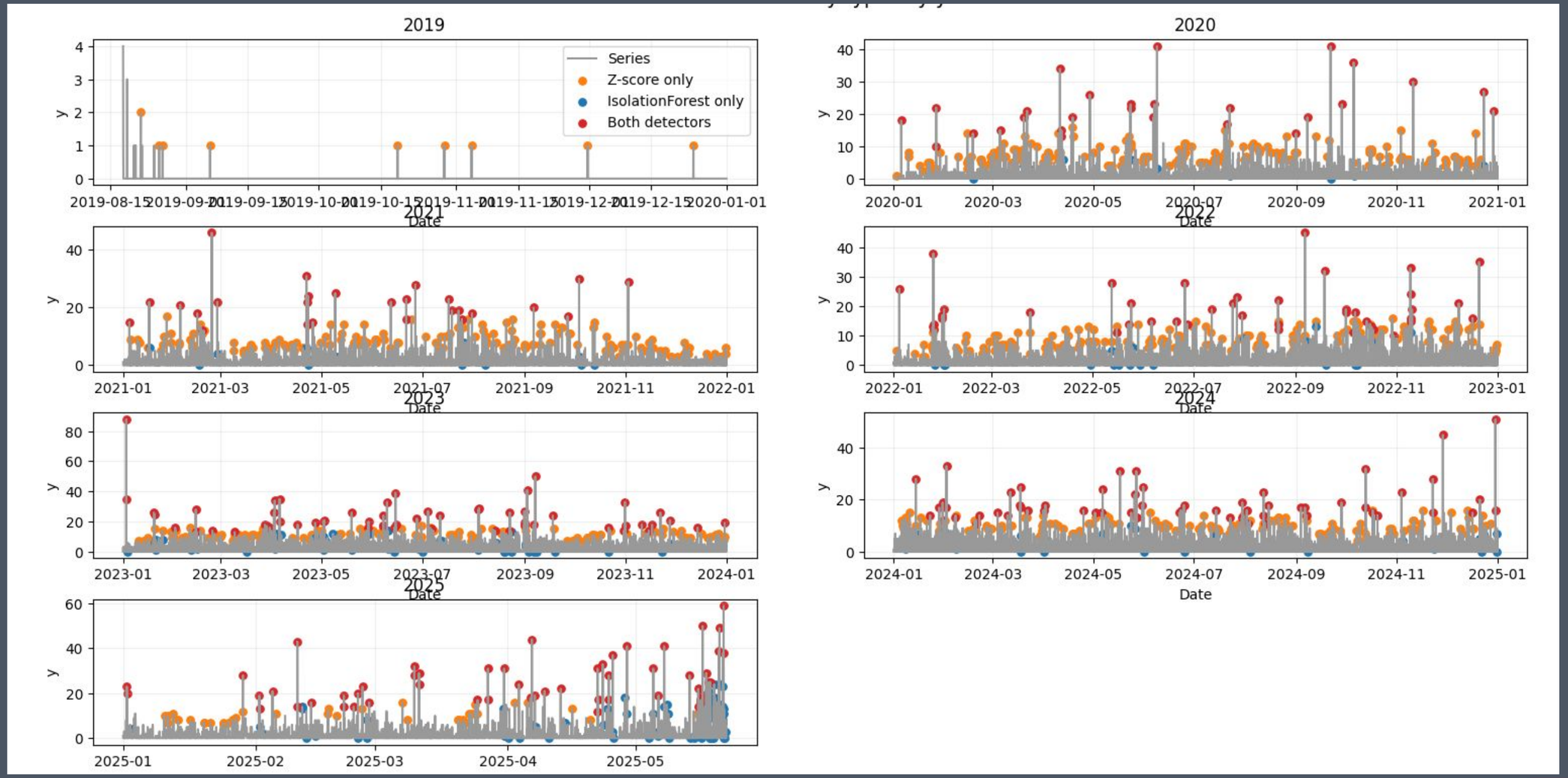
X = X.fillna(method="bfill").fillna(0)

model = IsolationForest(
    n_estimators=300,
    contamination=contamination,
    random_state=random_state,
    n_jobs=-1,
)
model.fit(X)

scores = model.decision_function(X)   # lower = more anomalous
pred = model.predict(X)               # -1 anomalous, 1 normal

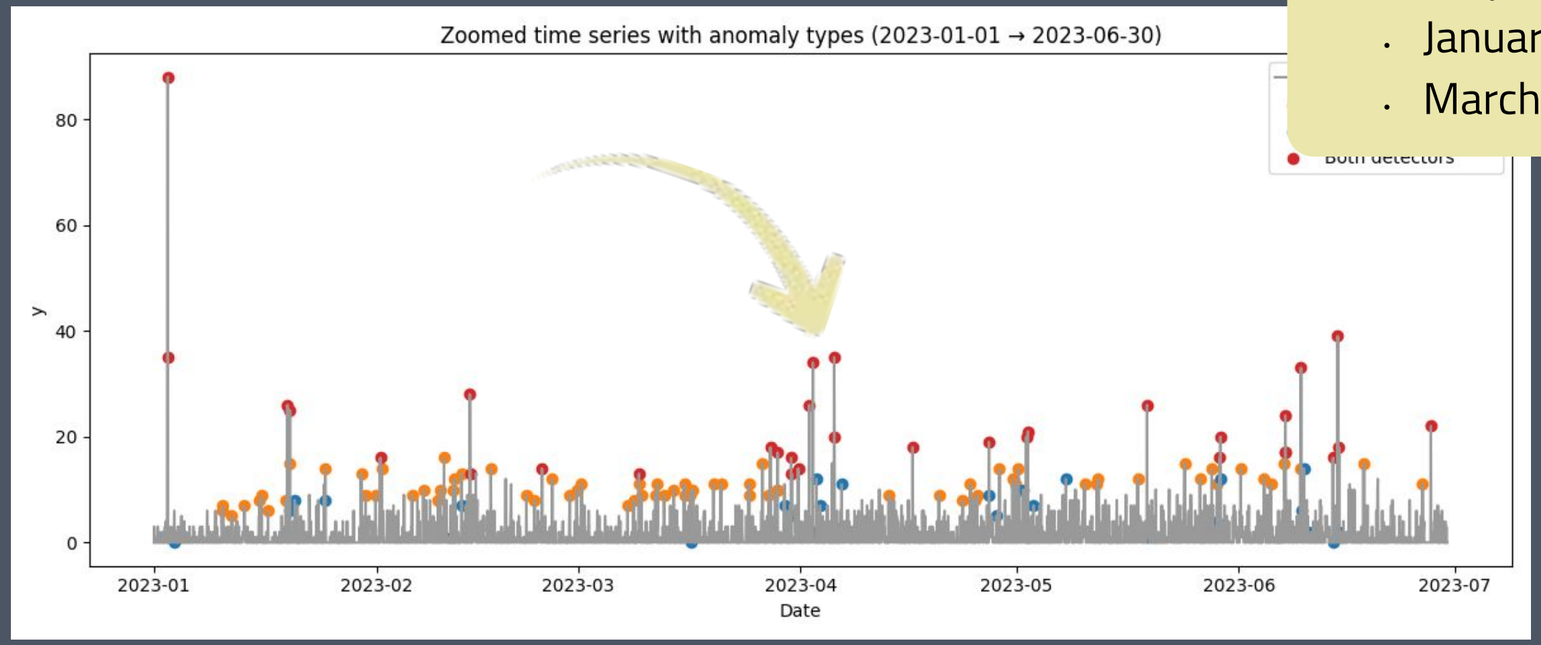
df["iso_score"] = scores
df["is_anomaly_iso"] = (pred == -1).astype(int)
return df
```

03 - Anomaly Detection



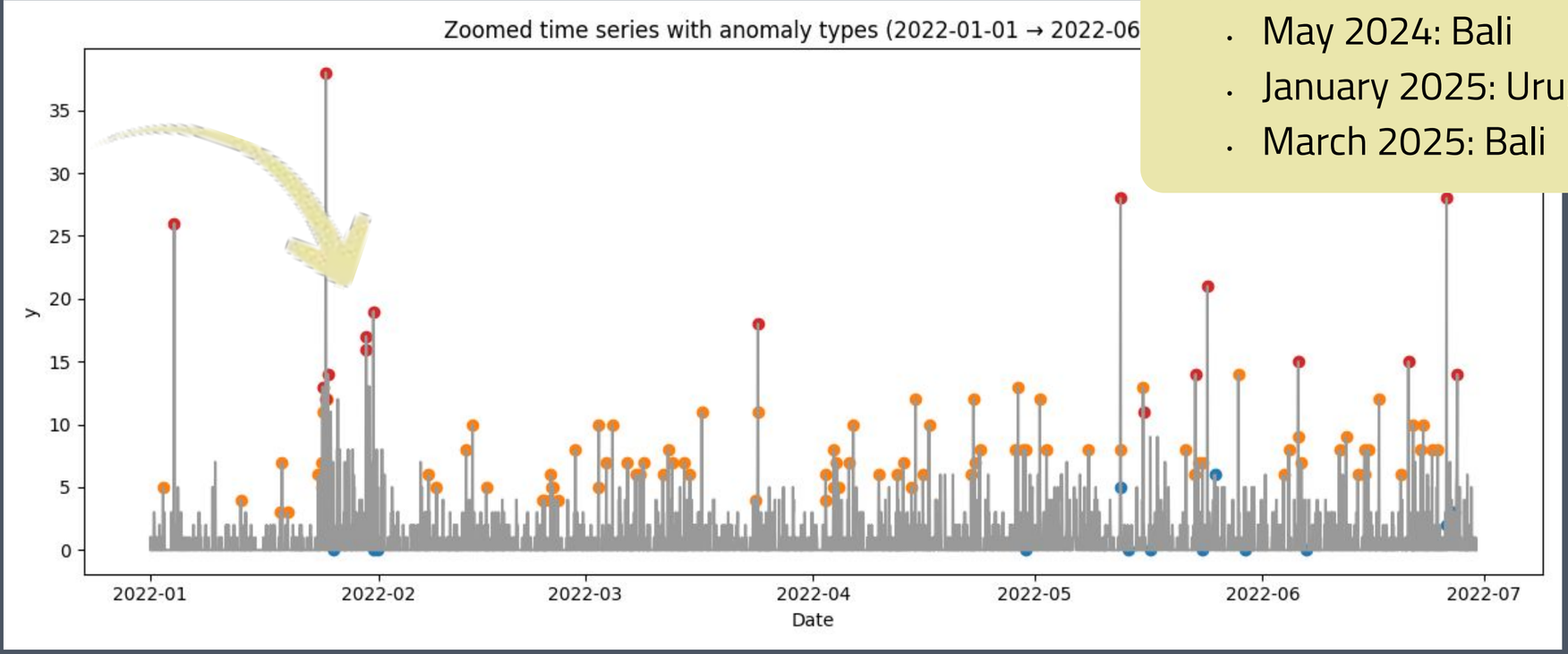
03 - Anomaly Detection

- May 2021: I leave Madrid and become a digital nomad
- Spring 2021: Lanzarote
- Summer 2021: Mexico
- September 2021: Changed jobs
- October 2021: Sitges
- September 2022: Bali
- April 2023: Bali - Divemaster
- May 2024: Bali
- January 2025: Uruguay-Brazil-Argentina
- March 2025: Bali



03 - Anomaly Detection

- May 2021: I leave Madrid and become a digital nomad
- Spring 2021: Lanzarote
- Summer 2021: Mexico
- September 2021: Changed jobs
- October 2021: Sitges
- September 2022: Bali
- April 2023: Bali - Divemaster
- May 2024: Bali
- January 2025: Uruguay-Brazil-Argentina
- March 2025: Bali



04 - Event Forecasting

NHITS (Nixtla): Neural Hierarchical Interpolation for Time Series forecasting

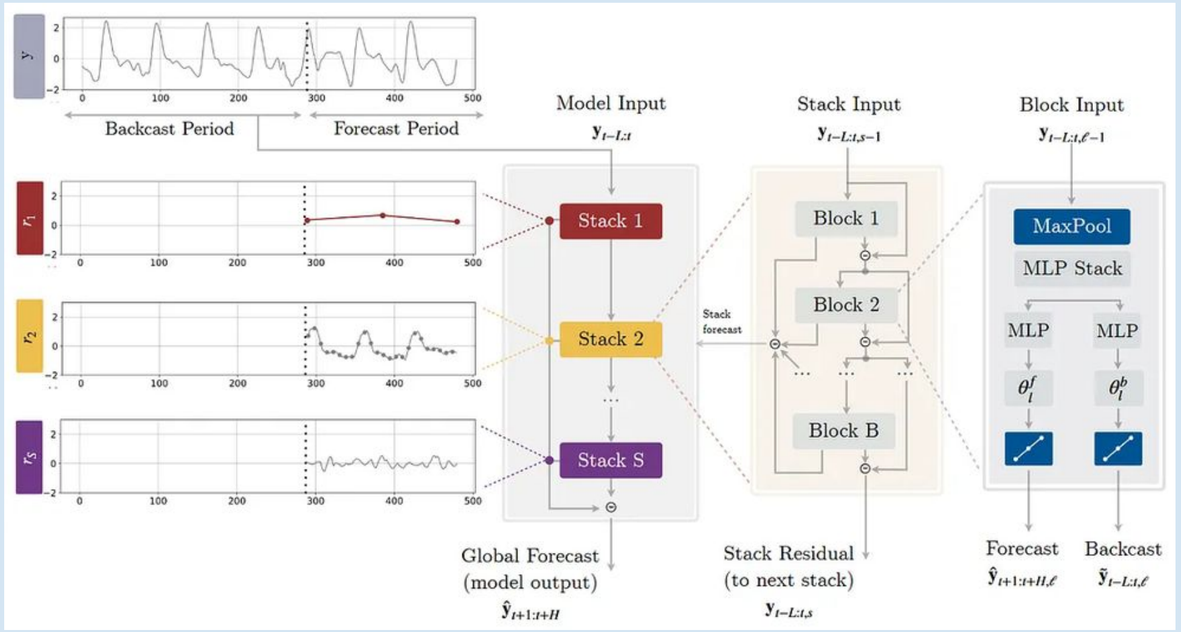
- An evolution of N-BEATS, optimised for forecasting.
- Advantages:
  - Lower memory consumption and computational cost.
  - More efficient than TimesNet or PatchTST.
  - Faster training on small datasets: ideal for our use case.



nixtla

*https://medium.com/data-science/n-hits-making-deep-learning-for-time-series-forecasting-more-efficient-d00956fc3e93

03 - Event Forecasting



Input. Time series split into:

- Backcast: window of historical data.
- Forecast: future points to predict.

Hierarchical stacks. They process the series at different levels of detail:

- Coarse: global patterns.
- Intermediate: adds more precision.
- Fine: corrects noise and small patterns.

Blocks within each stack. Each block produces two outputs:

- Backcast: reconstructs part of the past (the explained portion is removed, so the next block works on the residual).
- Forecast: contributes a partial prediction of the future.

*https://medium.com/data-science/n-hits-making-deep-learning-for-time-series-forecasting-more-efficient-d00956fc3e93

03 - Event Forecasting: NHITS

Train: the entire time series except 2025
Test: 2025 up to May
Model with baseline parameters
Model aggregated by day
Forecast horizon: 142 days

```python
def train_and_forecast_nhits(ts_df, input_size=30, max_steps=900):

    ts_df = ts_df.copy()
    ts_df['ds'] = pd.to_datetime(ts_df['ds'])

    # Aggregate to daily totals
    ts_df['date'] = ts_df['ds'].dt.date
    daily_df = (
        ts_df.groupby('date')['y']
        .sum()
        .reset_index()
        .rename(columns={'date': 'ds'})
    )

    # Add a common unique_id for NHITS
    daily_df['unique_id'] = 'total'
    daily_df = daily_df[['unique_id', 'ds', 'y']]

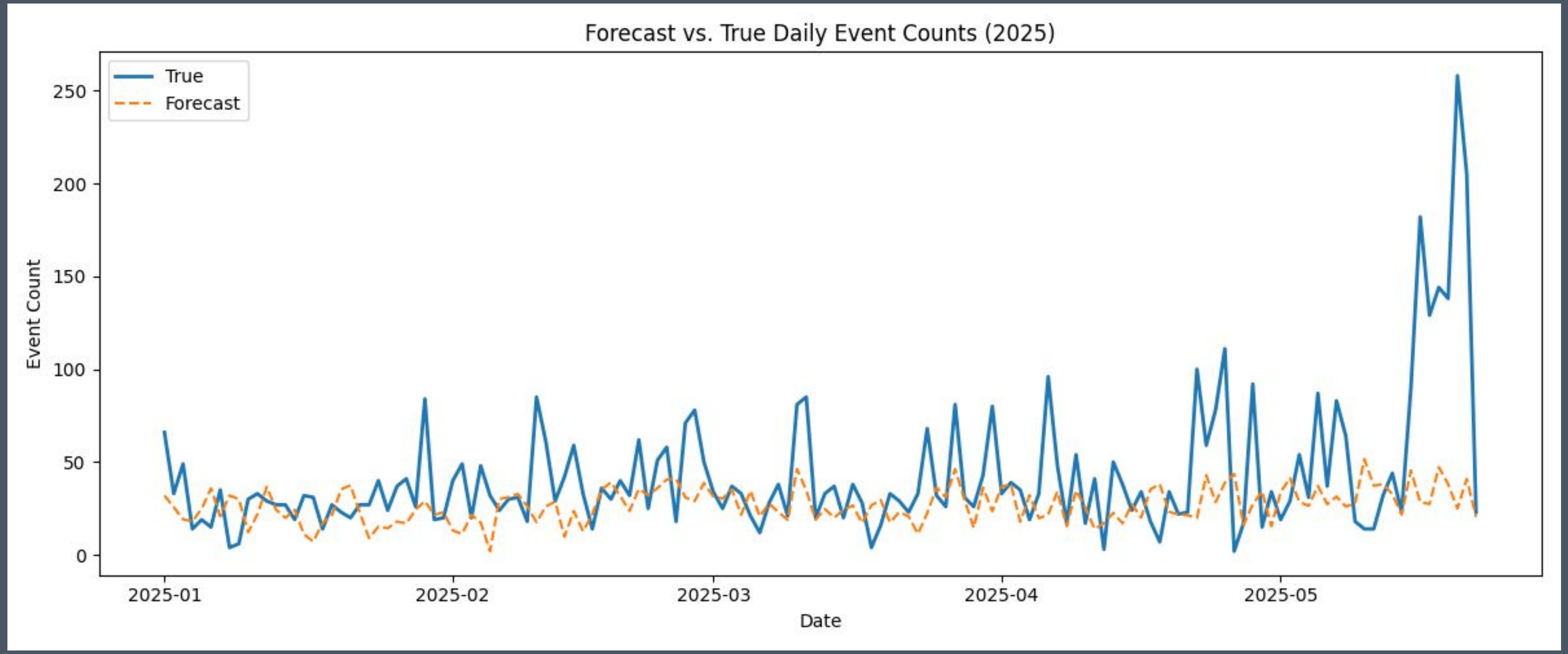
    # Split into training and test
    daily_df['ds'] = pd.to_datetime(daily_df['ds'])
    train_df = daily_df[daily_df['ds'] < '2025-01-01']
    true_2025_df = daily_df[(daily_df['ds'] >= '2025-01-01') & (daily_df['ds'] < '2026-01-01')]

    h = true_2025_df['ds'].nunique()

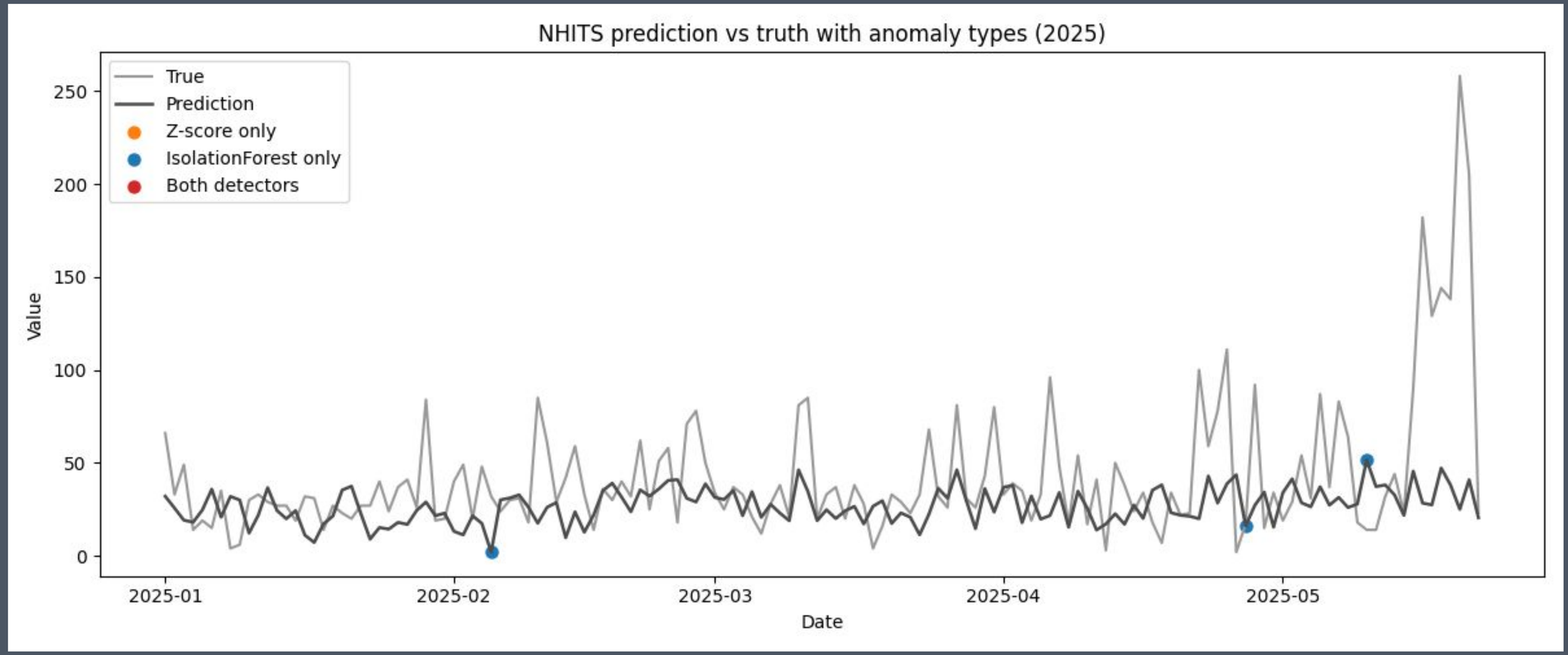
    model = NHITS(h=h, input_size=input_size, max_steps=max_steps)
    nf = NeuralForecast(models=[model], freq='D')
    nf.fit(df=train_df)
    forecast_df = nf.predict()

    return nf, forecast_df, true_2025_df
```

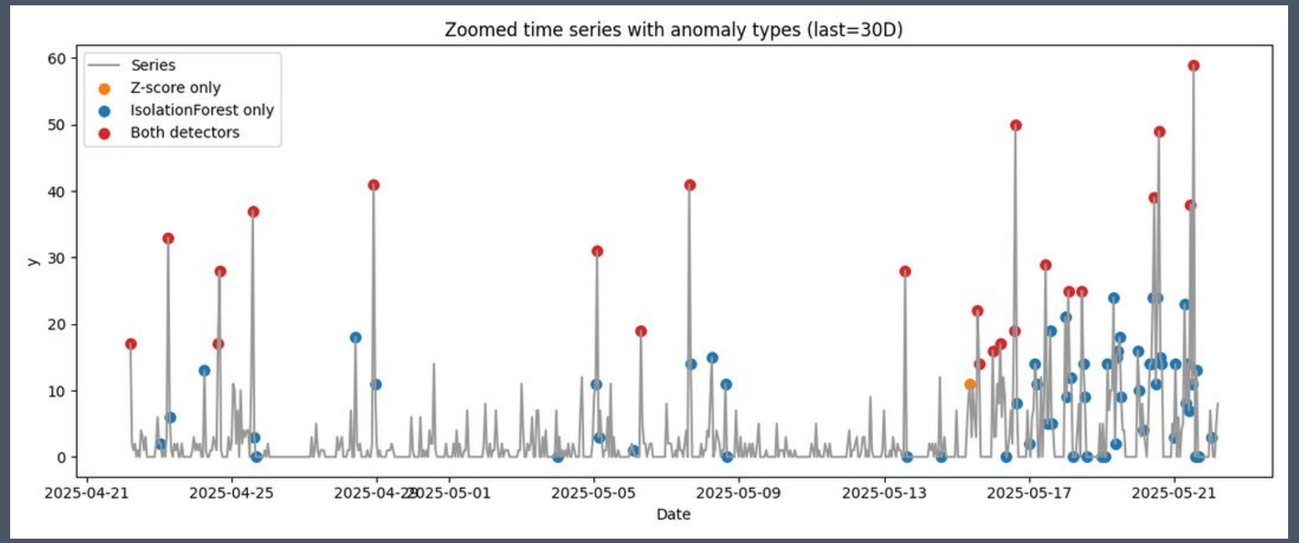
03 - Event Forecasting: NHITS

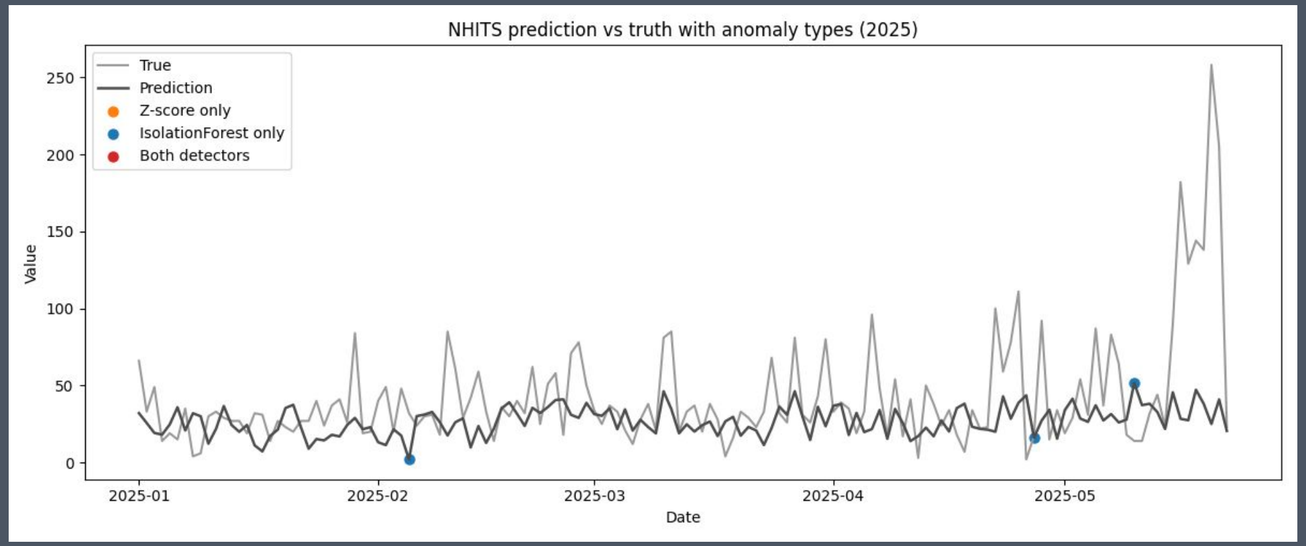


03 - Event Forecasting: NHITS



03 - Event Forecasting: NHITS





05- Instagram's Business

“Las mujeres ya no lloran las mujeres facturan”

But they are not the only ones making money...

- Instagram generated an estimated revenue of USD **66.9 billion** in 2024.
- In its 2024 10-K, Meta reported $160.6B in advertising revenue out of $164.5B total revenue: about 97.6% of all income.
- More than 60% of Instagram users are under 35 years old.



*https://www.edpb.europa.eu/news/news/2023/facebook-and-instagram-decisions-important-impact-use-personal-data-behavioural_en
https://www.businessofapps.com/data/instagram-statistics
https://www.sec.gov/Archives/edgar/data/1326801/000132680125000017/meta-20241231.htm

05- Instagram's Business

~~“Las mujeres ya no lloran las mujeres facturan”~~

“Instagram ya no factura, Instagram llora”

GDPR(LPD): In 2023 Instagram was fined heavily and subsequently introduced the option to pay for an ad-free personalised experience. The free version requires users to consent to the use of their data. However, this measure remains controversial because the EU questions whether users are genuinely consenting to the use of their data or are being coerced into doing so.

AI Act: it regulates the use rather than ownership of data, and Instagram’s advertising algorithm may be considered a general-purpose AI system with systemic risk due to the scale of its user base.



Almudena Barreiro

THANK YOU!

¿?

in/almudena-barreiro

github.com/AlmuBC/instagram-behavior-analysis